NLA project

This assessment is marked out of 100 and comprises 12% of the final course mark.

Your project takes the form of a single Jupyter notebook, using this notebook as model. Your answers to theoretical questions and your discussions of results should be entered in Markdown cells. Mardown cells support basic LaTex and you should use this for formulas. (To edit a cell, double click on it; to render it, press shift+enter or run.) The rest should be code cells and their output. Your code must generate and display all relevant output when run.

Submission

The project is **due by 4 pm on Friday November 14th 2025** (Week 9), to be submitted on Gradescope (DO NOT submit on Noteable).

Follow these instructions scrupulously:

1. To mimise the size of your notebook, do not include this preamble or the questions (you can start adding material to this notebook, then remove the preamble and questions to leave only your (numbered) answers, code and output).

2. When your notebook is ready, **run all the cells to make sure that the output is updated and displayed**.

3. Save a pdf version of the entire notebook. This shoud be a printable version in a single file, with individual pages (**not a single long page**). To obtain this, the safest way is in three steps:

   - Produce a html version using nbconvert. In Noteable Jupyter Lab, this is done using 'File > Save and export notebook as > HTML'. Alternatively, in a terminal, type `jupyter nbconvert --to html filename.ipynb`

   with your file name as 'filename'.

- Download the html file and open it in your browser.
- In your browser, use 'print as pdf' or similar to save a **multi-page** pdf version. (In Chrome, use 'print' and set 'destination' to 'save as pdf'). Use a 'Lanscape' layout for the convenience of the markers.

Avoid long lines of code which can get truncated in the pdf (you can write python statements over multiple lines using a backslash at the end of each line).

3. **Submit your pdf on Gradescope** using the assignment 'Project (your pdf file)'. Please **tag your pages** with the relevant questions in Gradescope.

4. **Submit also your Jupyter .ipynb file on Gradescope** using the other assignment 'Project (Jupyter notebooks, your .ipynb files only)'. We will mark the pdf files and use only the .pynb in case of issues with the pdf.

Answer 1.1 |

(a) ⇒ (b):

Suppose $A = QR$ where R has positive diagonal entries.
Then
$A^T A = (QR)^T (QR) = R^T R$,
so this gives the Cholesky factor of $A^T A$.
Since $A = QR$, multiplying from the right by R^{-1} gives
$Q = AR^{-1}$.

(b) ⇒ (a):

Now assume $A^T A = R^T R$ and $Q = AR^{-1}$.
Then
$Q^T Q = (AR^{-1})^T (AR^{-1}) = (R^{-1})^T (A^T A) R^{-1} = (R^{-1})^T (R^T R) R^{-1} = I$.
So Q is orthogonal.
Because $Q = AR^{-1}$, multiplying both sides by R gives $QR = A$.

Answer 1.2

In [28]:
```
import numpy as np
import scipy as sc
import matplotlib.pyplot as plt

def chol(A):
    """
    Compute the Cholesky factorisation of a symmetric positive definite matrix A.
    Returns upper triangular matrix R such that A = R^T R.
    """
    n = A.shape[0]  # Get matrix dimension
    R = np.copy(A).astype(float)  # Initialize R as a copy of A

    for k in range(n):  # Loop over columns
        for j in range(k + 1, n):  # Loop over rows below diagonal
            alpha = R[k, j] / R[k, k]  # Compute multiplier
            R[j, j:] = R[j, j:] - alpha * R[k, j:]  # Update row j
            R[j, k] = 0  # Zero out below diagonal entry
        R[k, k:] = R[k, k:] / np.sqrt(R[k, k])  # Scale row k

    return R

# test code on small example
A = np.array([[1, 1, 1], [1, 2, 2], [1, 2, 3]], dtype=float)
R = chol(A)
print(R)
```

```
[[1. 1. 1.]
 [0. 1. 1.]
 [0. 0. 1.]]
```

Answer 1.3

In [29]:
```
# Using forward substitution for matrix equations from Week 3
def FS(L, B):
    '''
    Use FS to solve LY = B.
    '''

    N = B.shape  # Get shape of right-hand side
    Y = np.zeros(N)  # Allocate solution array

    if len(N) == 1:  # Vector case
        for j in range(N[0]):
            Y[j] = (B[j] - L[j, :j] @ Y[:j]) / L[j, j]  # Forward elimination
    else:  # Matrix case
        for j in range(N[0]):
            Y[j, :] = (B[j, :] - L[j, :j] @ Y[:j, :]) / L[j, j]  # Column-by-column

    return Y

def qr(A):
    """
    QR factorisation via Cholesky method.
    Computes A = QR where Q is orthogonal, R upper triangular.
    """
    ATA = A.T @ A  # Form Gram matrix
    R = chol(ATA)  # Cholesky: A^T A = R^T R
    Y = FS(R.T, A.T)  # Solve R^T Y = A^T
    Q = Y.T  # Transpose to obtain Q

    return Q, R

# Test on provided example
A = np.array([[1, 2, 0], [0, 1, 1], [1, 0, 1]], dtype=float)
Q, R = qr(A)
print(Q)
print(R)
```

```
[[ 0.70710678  0.57735027 -0.40824829]
 [ 0.          0.57735027  0.81649658]
 [ 0.70710678 -0.57735027  0.40824829]]
[[1.41421356 1.41421356 0.70710678]
 [0.         1.73205081 0.        ]
 [0.         0.         1.22474487]]
```

Answer 1.4

Computing the product $A^T A$ takes n^3 operations.
Cholesky factorisation takes about
$n^3/3 + n^2 + 2n/3$.
Then forming $Q = AR^{-1}$ uses n triangular solves, about $n^3/2$ work.

Altogether:
$C(n) = n^3 + n^3/3 + n^3/2 + O(n^2) = (11n^3)/6 + O(n^2)$.

Answer 2.1

Only–if direction :

If $(A+\Delta A)x = zx$ and $||\Delta A|| \leq \varepsilon$, then
$(A - zI)x = -\Delta A x.$
Rewriting gives
$x = -(A - zI)^{-1}\Delta A x.$
Taking norms,
$||x|| \leq ||(A - zI)^{-1}||\,||\Delta A||\,||x|| \leq ||(A - zI)^{-1}||\,\varepsilon\,||x||.$
Thus
$||(A - zI)^{-1}|| \geq 1/\varepsilon.$

If–direction:

Assume $||(A - zI)^{-1}|| \geq 1/\varepsilon.$
Then there exists a unit vector x with
$||(A - zI)^{-1}x|| = 1/\delta$ for some $\delta \leq \varepsilon.$
Let $(A - zI)^{-1}x = (1/\delta)y$ with $||y|| = 1.$
This gives $(A - zI)y = \delta x.$
Define $\Delta A = -\delta\, xy^*.$
Then $||\Delta A|| = \delta \leq \varepsilon$ and
$(A + \Delta A)y = Ay - \delta x = zy.$
So z becomes an eigenvalue of $A + \Delta A$

Answer 2.2

Proof If $A = Q\Lambda Q^T$ with orthonormal eigenvectors, then

$(A - zI)^{-1} = Q(\Lambda - zI)^{-1}Q^T.$

For a diagonal matrix,

$||(\Lambda - zI)^{-1}||_2 = \max_i(1/|\lambda_i - z|).$

Therefore

$||(A - zI)^{-1}||_2 = 1/(\min_i |\lambda_i - z|).$

Thus

$||(A - zI)^{-1}|| \geq 1/\varepsilon$

happens exactly when

$\min_i |\lambda_i - z| \leq \varepsilon$

Answer 2.3

In [30]:
```
import numpy as np

def NoI(N, delta):
    """
    Algorithm NoI: compute 2-norm of matrix inverse.
    Uses inverse iteration on B^* B to find smallest eigenvalue.
    """
    n = N.shape[0]  # Matrix size
    x = np.ones(n, dtype='complex128')  # Initial vector
    x = x / np.linalg.norm(x)  # Normalize

    lambda_old = np.inf  # Previous eigenvalue estimate
    lambda_new = np.linalg.norm(N @ x) ** 2  # Initial eigenvalue

    while np.abs(lambda_new - lambda_old) > delta:  # Check convergence
        lambda_old = lambda_new  # Store previous value
        BstarB = N.conj().T @ N  # Form B^* B
        x = np.linalg.solve(BstarB, x)  # Inverse iteration step
        x = x / np.linalg.norm(x)  # Normalize iterate
        lambda_new = np.linalg.norm(N @ x) ** 2  # Update eigenvalue

    s = 1.0 / np.sqrt(lambda_new)  # Compute norm from eigenvalue
    return s

# Test
B = np.array([[2, 4, 2-1j], [1j, -3, 4], [3, -2*1j, 1+3*1j]])
print(NoI(B, 1e-5))
print(np.linalg.norm(np.linalg.inv(B), 2))
```

```
0.43317248951163373
0.43317249335481733
```

Answer 2.4

Proof From the Schur form $A = QTQ^*$, we get
$A - zI = Q(T - zI)Q^*$.

Thus

$$(A - zI)^{-1} = Q(T - zI)^{-1}Q^*.$$

If x is any vector and $y = Q^*x$, then $||y|| = ||x||$.
Hence

$$||(A - zI)^{-1}x|| = ||(T - zI)^{-1}y||.$$

This shows

$$||(A - zI)^{-1}|| = ||(T - zI)^{-1}||$$

Answer 2.5

In [31]:
```
def FS(L, b):
    # Find dimension of L
    n = L.shape[0]

    # Initialise y to zero vector of correct length
    y = np.zeros(n,dtype='complex128')

    # loop over elements of y, starting from first one
    for j in range(n):
        y[j] = (b[j] - L[j, :j] @ y[:j]) / L[j, j] # compute y_j

    return y

def BS(U, y):
    # Find dimension of U
```

```python
    n = U.shape[0]

    # Initialise x to zero vector of correct length
    x = np.zeros(n,dtype='complex128')

    # loop over elements of x, starting from last one
    for j in range(n-1, -1, -1):
        x[j] = (y[j] - U[j, j+1:] @ x[j+1:]) / U[j, j] # compute x_j

    return x

def NoI_tr(T, delta):
    """
    Efficient norm of inverse for triangular matrices.
    Uses forward and back substitution instead of matrix inverse.
    """
    n = T.shape[0]  # Matrix dimension
    x = np.ones(n, dtype='complex128')  # Starting vector
    x = x / np.linalg.norm(x)  # Normalize

    lambda_old = np.inf  # Initialize previous eigenvalue
    lambda_new = np.linalg.norm(T @ x) ** 2  # Compute initial eigenvalue

    while abs(lambda_new - lambda_old) > delta:  # Convergence test
        lambda_old = lambda_new  # Save old value
        y = FS(T.conj().T, x)  # Solve T^* y = x via forward substitution
        x = BS(T, y)  # Solve T x = y via back substitution
        x = x / np.linalg.norm(x)  # Normalize result
        lambda_new = np.linalg.norm(T @ x) ** 2  # Recompute eigenvalue

    s = 1.0 / np.sqrt(lambda_new)  # Extract norm
    return s


# Test
```

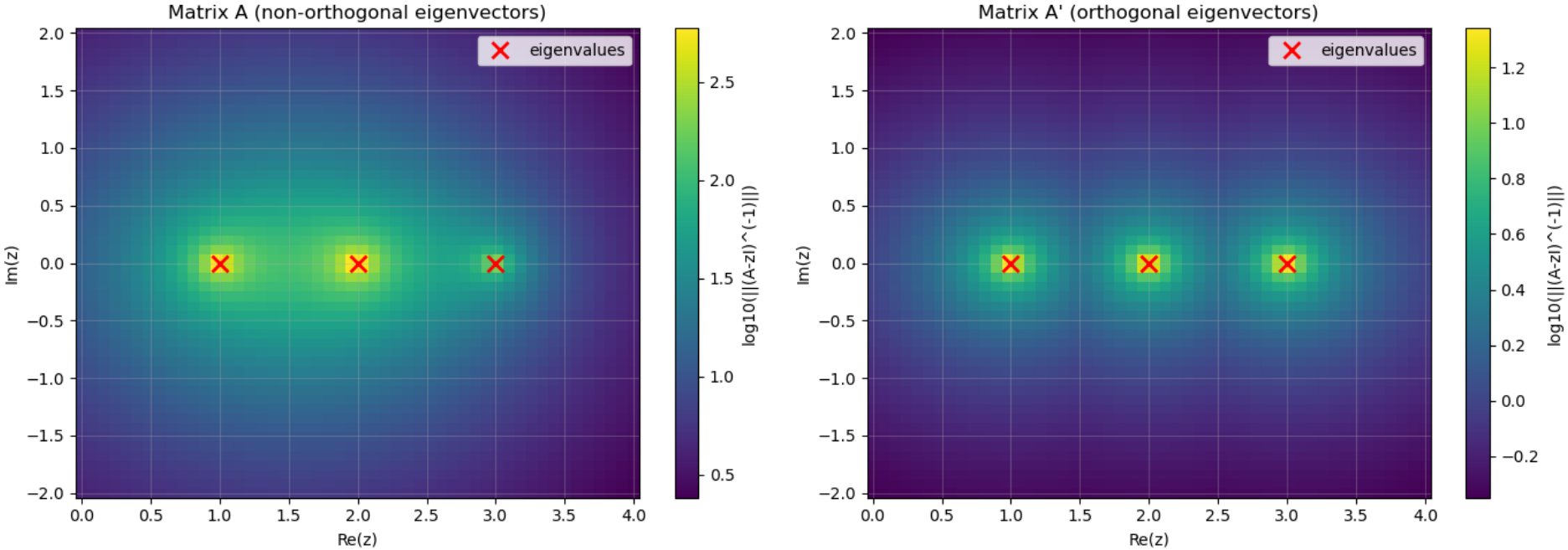
```
B = np.array([[2,4,2-1j],[0,-3,4],[0,0,1+3*1j]])
print(NoI(B,1e-5))
print(np.linalg.norm(np.linalg.inv(B),2))
```

```
1.5438059562262716
1.543805956230125
```

Answer 2.6

In [32]:
```
import scipy as sc
import matplotlib.pyplot as plt


def S_eps(A,delta,range_a,na,range_b,nb):
    n = A.shape[0]
    T, _ = sc.linalg.schur(A, output ='complex')
    a, b = np.meshgrid(np.linspace(range_a[0],range_a[1],na),
                       np.linspace(range_b[0],range_b[1],nb))
    inv_norm = np.zeros((na,nb))
    for i in range(na):
        for j in range(nb):
            z = a[i,j] + 1j * b[i,j]
            inv_norm[i, j] = NoI_tr(T - z * np.eye(n), delta)  # Compute ||(T - zI)^{-1}||

    return a, b, inv_norm

# Examples of 2 matrices A and A' with the same spectrum
P = np.array([[-1,2,3],[-1,2.2,3],[-1,1.8,2]])
A = np.linalg.inv(P) @ np.diag([1,2,3]) @ P
Pp, _ = np.linalg.qr(P)
Ap = np.linalg.inv(Pp) @ np.diag([1,2,3]) @ Pp

# Compute pseudospectra
a1, b1, inv_norm1 = S_eps(A, 1e-5, [0, 4], 50, [-2, 2], 50)
a2, b2, inv_norm2 = S_eps(Ap, 1e-5, [0, 4], 50, [-2, 2], 50)
```

```python
# Visualization
plt.figure(figsize=(14, 5))

# Plot matrix A
plt.subplot(1, 2, 1)
plt.pcolor(a1, b1, np.log10(inv_norm1), shading='auto', cmap='viridis')
plt.plot([1, 2, 3], [0, 0, 0], 'rx', markersize=10, markeredgewidth=2, label='eigenvalues')
plt.xlabel('Re(z)')
plt.ylabel('Im(z)')
plt.title('Matrix A (non-orthogonal eigenvectors)')
plt.grid(True, alpha=0.3)
plt.colorbar(label='log10(||(A-zI)^(-1)||)')
plt.legend()

# Plot matrix A'
plt.subplot(1, 2, 2)
plt.pcolor(a2, b2, np.log10(inv_norm2), shading='auto', cmap='viridis')
plt.plot([1, 2, 3], [0, 0, 0], 'rx', markersize=10, markeredgewidth=2, label='eigenvalues')
plt.xlabel('Re(z)')
plt.ylabel('Im(z)')
plt.title("Matrix A' (orthogonal eigenvectors)")
plt.grid(True, alpha=0.3)
plt.colorbar(label='log10(||(A-zI)^(-1)||)')
plt.legend()

plt.tight_layout()
plt.show()
```



Although A and A' share the same eigenvalues, the difference in their eigenvectors matters a lot.
Matrix A has eigenvectors that are far from orthogonal, so the quantity $||(A - zI)^{-1}||$ becomes large over wide regions, which signals strong sensitivity of the spectrum.
Matrix A' has orthogonal eigenvectors, so the large values only appear very close to each eigenvalue, and the eigenvalues are much more stable.

Answer 2.7

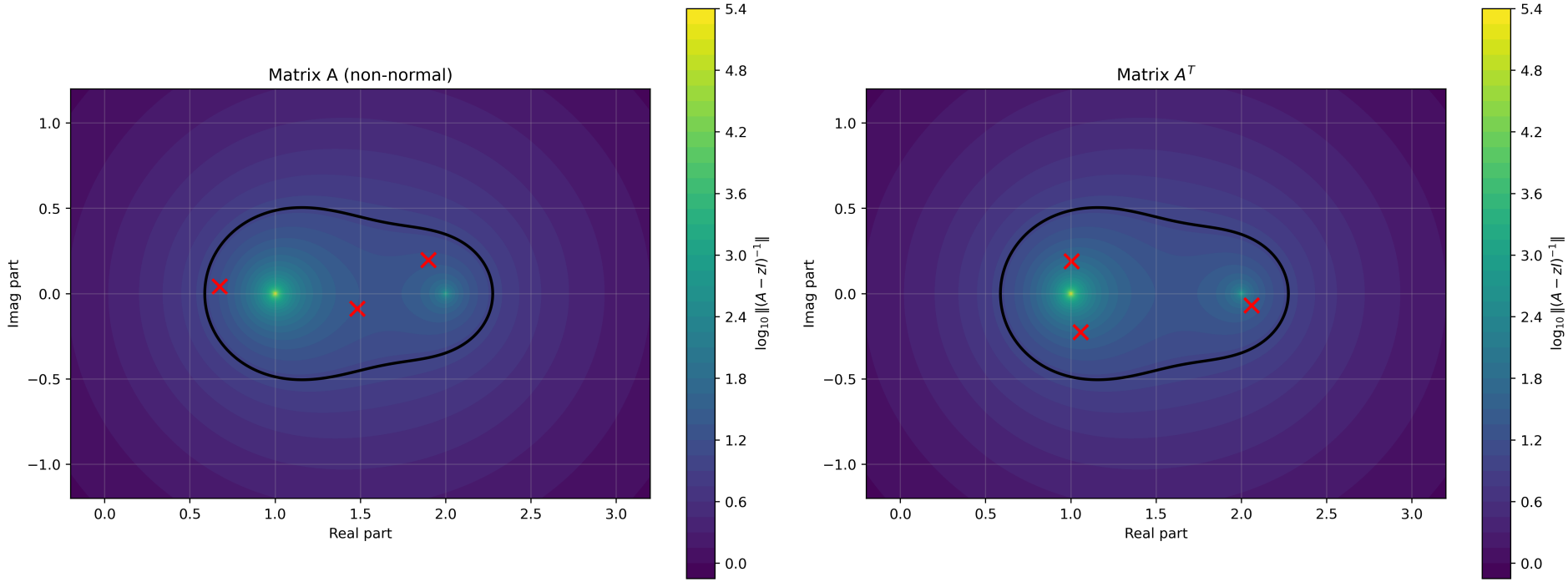
In [33]:
```
import numpy as np
import matplotlib.pyplot as plt
```

```python
def rand_mat_norm(n, norm):
    '''
    Generation of a random matrix of size n with given 2-norm
    '''
    svalues = np.random.rand(n)
    svalues = svalues / np.max(svalues) * norm
    Sigma = np.diag(svalues)

    U = np.random.randn(n, n) + 1j * np.random.randn(n, n)
    U, _ = np.linalg.qr(U)

    V = np.random.randn(n, n) + 1j * np.random.randn(n, n)
    V, _ = np.linalg.qr(V)

    return U @ Sigma @ V.T


def pseudo_plot(A, eps=0.1, res=420, title="", margin=1.2):
    """
    Produce a pseudospectrum-style plot for a matrix A.
    The coloured background represents log10(||(A - zI)^{-1}||),
    and the red crosses mark eigenvalues of A after a perturbation
    of size <= eps.
    Returns the 2-norm of the perturbation used.
    """

    n = A.shape[0]
    I = np.eye(n)

    # eigenvalues determine plotting window
    vals = np.linalg.eigvals(A)
    xr = np.linspace(vals.real.min() - margin, vals.real.max() + margin, res)
    yr = np.linspace(vals.imag.min() - margin, vals.imag.max() + margin, res)
    X, Y = np.meshgrid(xr, yr)
    Z = X + 1j*Y
```

```python
# grid for 1 / minimum singular value
R = np.zeros_like(X)
for p in range(res):
    for q in range(res):
        M = A - Z[p, q] * I
        sm = np.linalg.svd(M, compute_uv=False)[-1]   # smallest singular value
        if sm > 0:
            R[p, q] = 1.0 / sm
        else:
            R[p, q] = np.inf

# convert to log10 scale
logR = np.log10(R)

# draw filled contours
img = plt.contourf(X, Y, logR, levels=40, cmap="viridis")

# draw the pseudospectral boundary
plt.contour(X, Y, R, levels=[1/eps], colors="black", linewidths=2)

# create a perturbation and plot the resulting eigenvalues
Delta = rand_mat_norm(n, eps)
new_vals = np.linalg.eigvals(A + Delta)

plt.scatter(new_vals.real, new_vals.imag,marker="x", s=110, c="red", linewidths=2)

# formatting
ax = plt.gca()
ax.set_aspect("equal", "box")
plt.xlabel("Real part")
plt.ylabel("Imag part")
plt.title(title)
plt.grid(alpha=0.25)
```

```
    cb = plt.colorbar(img)
    cb.set_label(r"$\log_{10} \|(A - zI)^{-1}\|$")

    return np.linalg.norm(Delta, 2)


# Example usage
np.random.seed(0)

A = np.array([[2, 2, 1],
              [0, 1, 1],
              [0, 0, 1]], dtype=float)

eps = 0.10

plt.figure(figsize=(16, 6), dpi=400)

plt.subplot(1, 2, 1)
normA = pseudo_plot(A, eps=eps, title="Matrix A (non-normal)")
print("||ΔA||_2 for A =", normA)

plt.subplot(1, 2, 2)
normAT = pseudo_plot(A.T, eps=eps, title="Matrix $A^T$")
print("||ΔA||_2 for A^T =", normAT)

plt.tight_layout()
plt.show()
```

```
||ΔA||_2 for A = 0.09999999999999999
||ΔA||_2 for A^T = 0.1
```



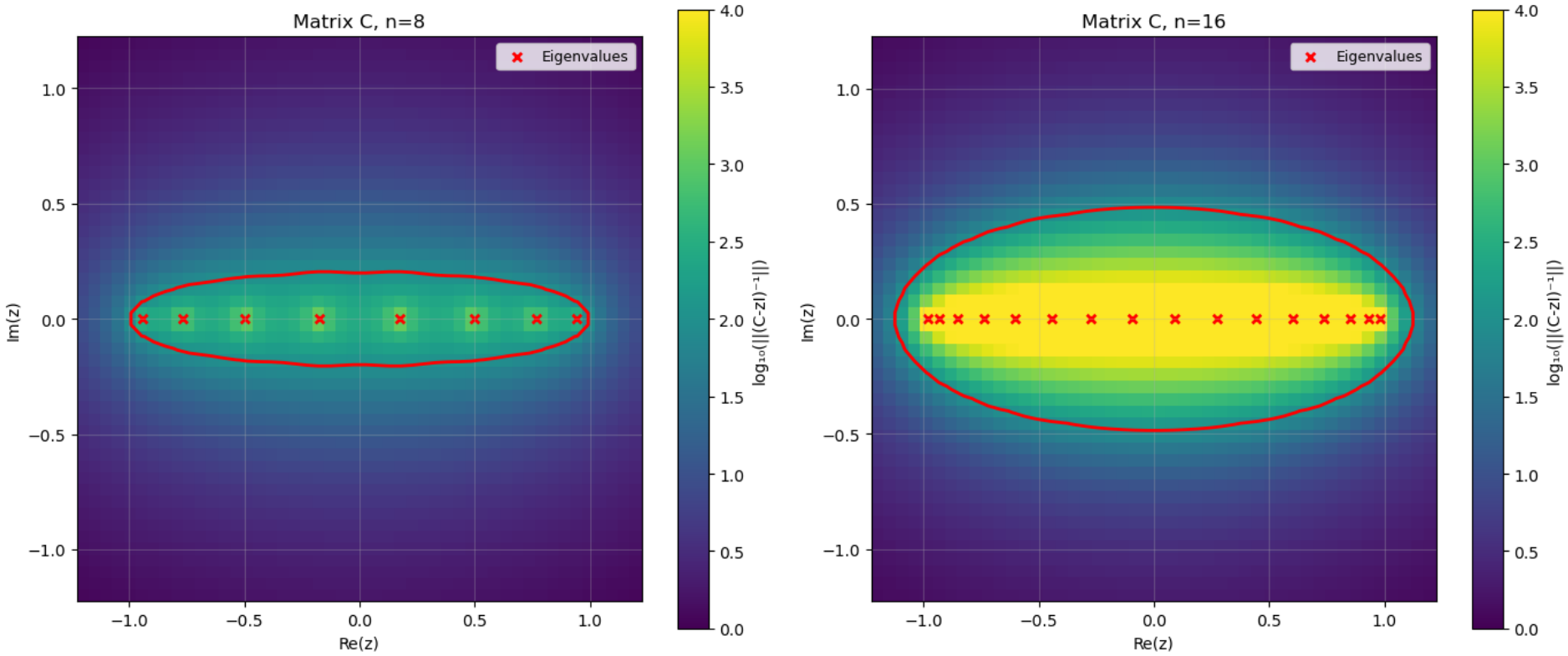
The red contour corresponds to $||(A - zI)^{-1}|| = 1/\varepsilon$.
For the first matrix, the perturbed eigenvalues spread out across the interior of this contour.
For the second matrix, the eigenvalues barely move and remain near their original locations.
All perturbed eigenvalues stay inside the contour in both cases, which matches the statement proved earlier in Question 2.1

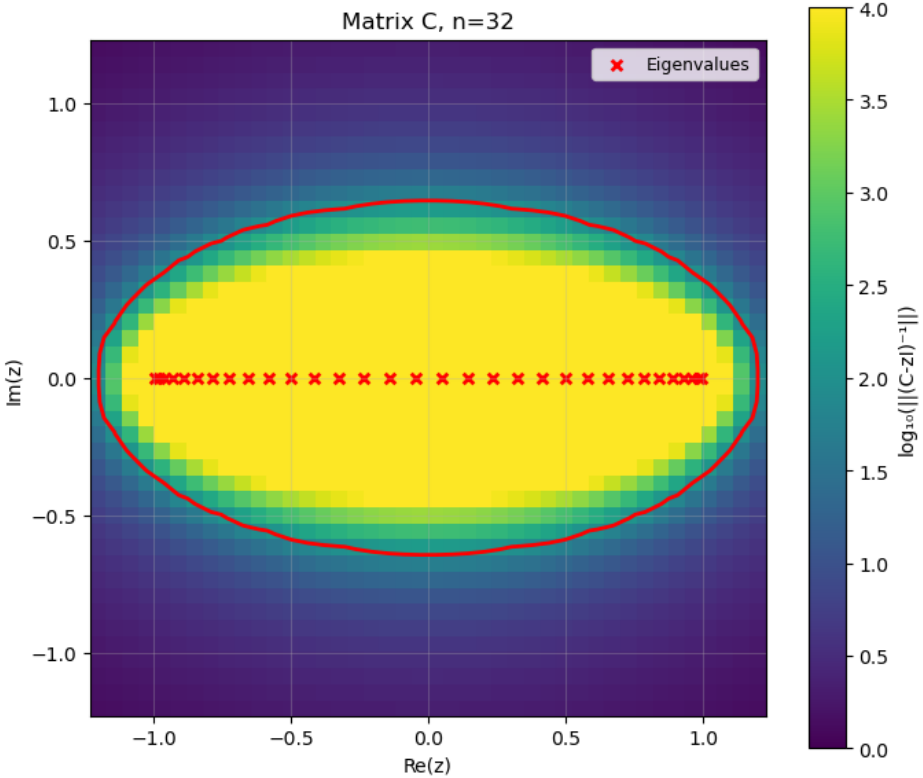
Answer 2.8

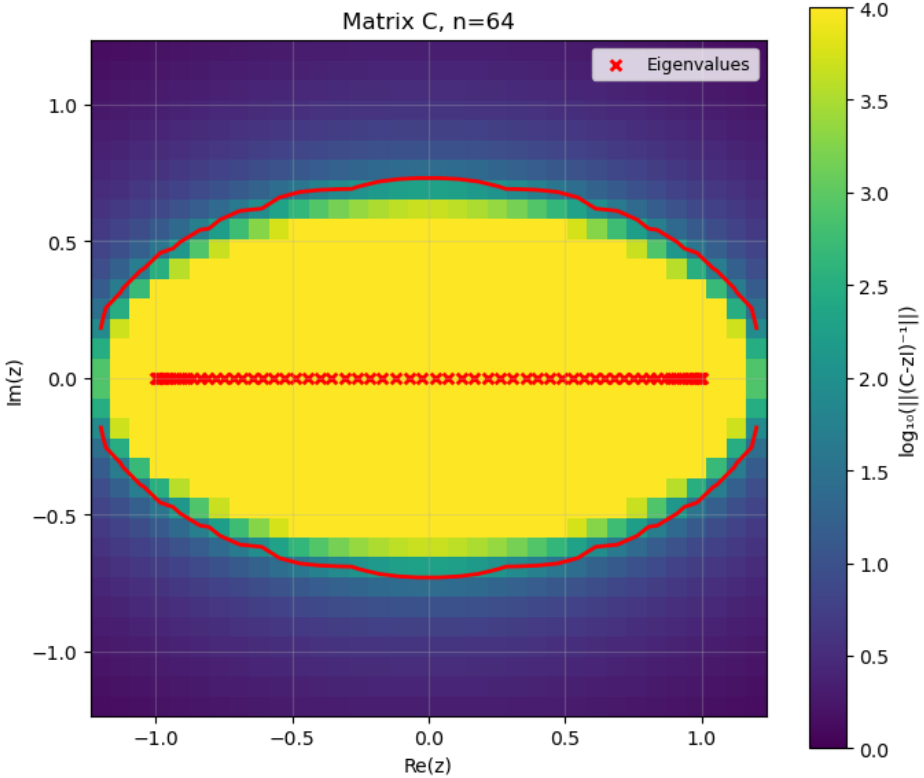
In [34]:
```
import numpy as np
import scipy as sc
import scipy.linalg
import matplotlib.pyplot as plt

def mat_T(n):
```

```python
    """Generate the Toeplitz matrix C."""
    c = np.append(np.array([0, 0.25]), np.zeros(n-2))  # First column
    r = np.append(np.array([0, 1]), np.zeros(n-2))  # First row
    return sc.linalg.toeplitz(c, r)  # Construct Toeplitz matrix

# Analyze different n-sizes
sizes = [8, 16, 32, 64]
epsilon_small = 0.01

fig, axes = plt.subplots(2, 2, figsize=(14, 12))
axes = axes.flatten()

for idx, n in enumerate(sizes):
    C = mat_T(n)  # Generate matrix
    eigenvalues_C = np.linalg.eigvals(C)  # Compute eigenvalues

    # Set plotting range based on eigenvalues
    a_min = min(-1.2, eigenvalues_C.real.min() - 0.2)
    a_max = max(1.2, eigenvalues_C.real.max() + 0.2)
    b_min = min(-1.2, eigenvalues_C.imag.min() - 0.2)
    b_max = max(1.2, eigenvalues_C.imag.max() + 0.2)

    # Compute the sensitivity on grid (coarser for larger matrices)
    grid_size = max(30, 50 - n//4)  # Adaptive grid size
    a_C, b_C, inv_norm_C = S_eps(C, 1e-4, [a_min, a_max], grid_size, [b_min, b_max], grid_size)

    # Plot graphs
    ax = axes[idx]
    pcm = ax.pcolor(a_C, b_C, np.log10(inv_norm_C), shading='auto', cmap='viridis', vmin=0, vmax=4)

    # Try to plot contour at 1/epsilon
    try:
        ax.contour(a_C, b_C, inv_norm_C, levels=[1/epsilon_small], colors='red', linewidths=2)
    except:
        pass  # Contour may fail if values don't reach threshold
```

```
    ax.scatter(eigenvalues_C.real, eigenvalues_C.imag, c='red', s=30, marker='x',
               linewidth=2, label='Eigenvalues')
    ax.set_xlabel('Re(z)')
    ax.set_ylabel('Im(z)')
    ax.set_title(f'Matrix C, n={n}')
    ax.legend(fontsize=9)
    ax.grid(True, alpha=0.3)
    ax.set_aspect('equal', adjustable='box')
    plt.colorbar(pcm, ax=ax, label='log₁₀(||(C−zI)⁻¹||)')

plt.tight_layout()
plt.show()
```




Matrix C, n=32
Eigenvalues
Im(z)
Re(z)
log₁₀(||(C-zI)⁻¹||)


Matrix C, n=64
Eigenvalues
Im(z)
Re(z)
log₁₀(||(C-zI)⁻¹||)

In [35]:
```
# We test for eigenvalue movement under perturbations
print("Eigenvalue movement analysis for matrix C")
print(f"Testing with perturbation size ε = {epsilon_small}\n")

for n in [8, 16, 32, 64]:
    C = mat_T(n)  # Generate the matrix
    original_eigs = np.linalg.eigvals(C)  # Original eigenvalues

    # Generate multiple perturbations and track maximum movement
    max_movements = []
    for _ in range(10):  # Average over 10 random perturbations
        Delta = rand_mat_norm(n, epsilon_small)  # Generate perturbation
        perturbed_eigs = np.linalg.eigvals(C + Delta)  # Perturbed eigenvalues

        # Compute maximum distance any eigenvalue moved
        max_movement = 0
        for p_eig in perturbed_eigs:
            min_dist = np.min(np.abs(original_eigs - p_eig))  # Distance to nearest original
            max_movement = max(max_movement, min_dist)
        max_movements.append(max_movement)

    avg_max_movement = np.mean(max_movements)
    ratio = avg_max_movement / epsilon_small

    print(f"n={n:d}: Max eigenvalue movement = {avg_max_movement:.6f}")
    print(f"       Ratio (movement/ε) = {ratio:.2f}")
    print()
```

```
Eigenvalue movement analysis for matrix C
Testing with perturbation size ε = 0.01

n=8: Max eigenvalue movement = 0.071729
       Ratio (movement/ε) = 7.17

n=16: Max eigenvalue movement = 0.351050
       Ratio (movement/ε) = 35.11

n=32: Max eigenvalue movement = 0.535055
       Ratio (movement/ε) = 53.51

n=64: Max eigenvalue movement = 0.647316
       Ratio (movement/ε) = 64.73
```

The Toeplitz matrix C shows very strong sensitivity. We can see from our calculations above as well, as n increases, movement/e increases drastically. Its eigenvalues tend to lie close to the unit circle, and when n increases they become tightly packed.
The pseudospectrum grows rapidly with n, and $||(C - zI)^{-1}||$ gets large over a big region, not just around each eigenvalue.
Small perturbations of size 0.01 cause eigenvalues to move by much more than that amount, and the amplification becomes more severe for larger n.
This happens because C is highly non-normal and its eigenvectors are nearly linearly dependent, so C is extremely unstable with respect to its eigenvalues.

In [ ]: